COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 031	**SECTION:**	CPE21S3
DATE PERFORMED:	09/13/2026	**DATE SUBMITTED:**	09/13/2026
NAME:	Troy Jerron A. Bautista	**INSTRUCTOR:**	Sir Jimlord Quejado

ACTIVITY NO. 8.2 **Sorting Algorithms Pt2**

1. PROCEDURE OUTPUT

Code + Console Screenshot	**Main Code:**
	6 const int size = 100;
	7 void displaySort (int array[]);
	8
	9 int main() {
	10 int num[size];
	11 srand(time(0));
	12 for (int i = 0; i < size; i++) {
	13 num[i] = rand() % 100;
	14 }
	15 std::cout <<"100 randomly generated numbers: " <<std::endl;
	16 displaySort(num);
	Output:
	100 randomly generated numbers:
	22 57 65 35 0 40 14 21 99 45 38 84 85 49 80 32 21 64 26 48 97 29 44 1 73 89 32 62 6 44
	72 15 87 56 56 65 27 25 81 60 35 69 11 82 73 20 82 83 66 50 98 73 48 96 37 91 47 73 43
	16 76 18 97 37 26 72 3 4 29 76 82 14 44 67 54 11 43 55 11 63 13 2 27 68 42 17 75 23 16
	16 53 47 44 36 22 40 55 85 59 84

Observations	This code displays 100 random generated numbers into the console. With the use of srand, it gets the seed of the randomly generated number, and using rand we are able to randomize the generated numbers, but added a modulo of 100 to keeps the numbers from 1-100.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	**Header File:**
	5 template <typename T>
	6 void shellSort (T arr[], int size) {
	7 for (int gap = size / 2; gap > 0; gap /= 2) {
	8 for (int i = gap; i < size; i++) {
	9 int tempArr = arr[i];
	10 int j = i;
	11 while (j >= gap && arr[j - gap] > tempArr) {
	12 arr[j] = arr[j - gap];
	13 j -= gap;
	14 }
	15 arr[j] = tempArr;
	16 }
	17 }
	18 }

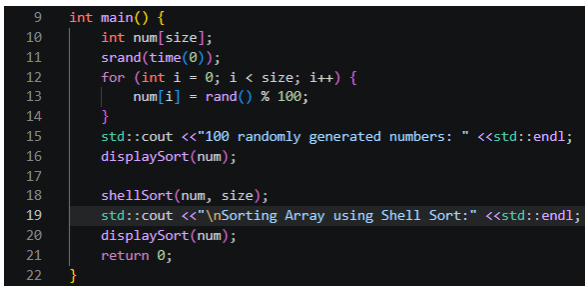
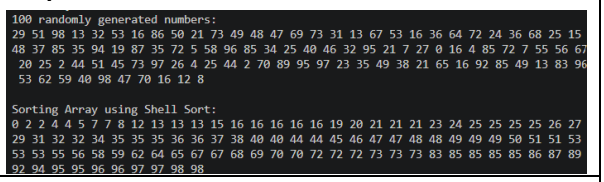
	Main Code:  **Output:** 
Observations	This code shows how shell sort works in coding. In the header file, it uses a nested for loop, one for the gap, and the other for iterating the whole list. The sort it by using the gap and shortening it until it does a bubble sort since, it will have no gap and th number will be adjacent to each other. The time complexity clearly depends, but it has a worst-case time of $O(n^2)$.

Table 8-2. Shell Sort Technique

Code + Console Screenshot

Header File:

```
20  template <typename T>
21  void merge (T arr[], int left, int middle, int right) {
22      int n1 = middle - left + 1;
23      int n2 = right - middle;
24
25      int leftArr[n1], rightArr[n2];
26
27      for (int i = 0; i < n1; i++) {
28          leftArr[i] = arr[left  + i];
29      }
30      for (int j = 0; j < n2; j++) {
31          rightArr[j] = arr[middle + 1 + j];
32      }
33
34      int i = 0, j = 0;
35      int k = left;
36
37      while (i < n1 && j < n2) {
38          if (leftArr[i] <= rightArr[j]) {
39              arr[k] = leftArr[i];
40              i++;
41          }
42          else {
43              arr[k] = rightArr[j];
44              j++;
45          }
46          k++;
47      }
```

```
48      while (i < n1) {
49          arr[k] = leftArr[i];
50          i++;
51          k++;
52      }
53      while (j < n2) {
54          arr[k] = rightArr[j];
55          j++;
56          k++;
57      }
58  }
59
60  void mergeSort (int arr[], int left, int right) {
61      if (left >= right)  {
62          return;
63      }
64
65      int middle = left + (right - left) / 2;
66
67      mergeSort(arr, left, middle);
68      mergeSort(arr, middle + 1, right);
69      merge(arr, left, middle, right);
70  }
```

	Main Code:
	`const int size = 100;`
	`void displaySort (int array[]);`
	
	`int main() {`
	`  int num[size];`
	`  srand(time(0));`
	`  for (int i = 0; i < size; i++) {`
	`  num[i] = rand() % 100;`
	`  }`
	`  std::cout <<"100 randomly generated numbers: " <<std::endl;`
	`  displaySort(num);`
	
	`  mergeSort(num, 0, size - 1);`
	`  std::cout <<"\nSorting Array using Merge Sort:" <<std::endl;`
	`  displaySort(num);`
	`  return 0;`
	`}`
	
	Output:
	`100 randomly generated numbers:`
	`66 6 45 89 59 61 95 20 96 45 92 37 96 27 47 96 13 34 90 10 28 46 69 44 77 70 32 53 64 9`
	`3 45 44 23 65 66 25 52 26 76 99 25 58 44 14 87 49 92 70 72 31 89 43 56 67 39 86 49 82 7`
	`0 40 30 49 13 42 41 95 64 33 0 6 1 58 38 76 82 80 70 9 12 81 68 39 56 63 39 64 47 64 57`
	` 43 66 96 60 86 8 17 17 3 10 51`
	
	`Sorting Array using Merge Sort:`
	`0 1 3 6 6 8 9 10 10 12 13 13 14 17 17 20 23 25 25 26 27 28 30 31 32 33 34 37 38 39 39 3`
	`9 40 41 42 43 43 44 44 44 45 45 45 46 47 47 49 49 51 52 53 56 56 57 58 58 59 60 61 6`
	`3 64 64 64 64 65 66 66 66 67 68 69 70 70 70 70 72 76 76 77 80 81 82 82 86 86 87 89 89 9`
	`0 92 92 93 95 95 96 96 96 96 99`

Observations	This code sorts the array by using the divide and conquer strategy to sort them one by one. It two functions, one to do the merging, the other is the main one to do the sorting. It uses two arrays, left array and right array to be used in the whole sorting process. The main file does the sorting and use the merge at the last part of the algorithm once the sublists have been sorted. The time complexity of this sorting algorithm is O(n log n) because of the strategy implemented.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	**Header File:**
	`template <typename T>`
	`int partition (T arr[], int low, int high) {`
	`  int pivot = arr[high];`
	`  int i = low - 1;`
	
	`  for (int j = low; j <= high - 1; j++) {`
	`  if (arr[j] < pivot) {`
	`  i++;`
	`  std::swap(arr[i], arr[j]);`
	`  }`
	`  }`
	`  std::swap(arr[i + 1], arr[high]);`
	`  return i + 1;`
	`}`
	
	`template <typename T>`
	`void quickSort (T arr[], int low, int high) {`
	`  if (low < high) {`
	`  int pivot = partition(arr, low, high);`
	`  quickSort(arr, low, pivot - 1);`
	`  quickSort(arr, pivot + 1, high);`
	`  }`
	`}`

	Main Code:
	`const int size = 100;`
	`void displaySort (int array[]);`
	
	`int main() {`
	`  int num[size];`
	`  srand(time(0));`
	`  for (int i = 0; i < size; i++) {`
	`  num[i] = rand() % 100;`
	`  }`
	`  std::cout <<"100 randomly generated numbers: " <<std::endl;`
	`  displaySort(num);`
	
	`  quickSort(num, 0, size - 1);`
	`  std::cout <<"\nSorting Array using Quick Sort:" <<std::endl;`
	`  displaySort(num);`
	`  return 0;`
	`}`
	
	Output:
	`100 randomly generated numbers:`
	`36 10 93 54 28 1 31 49 18 57 99 96 57 99 71 61 32 66 54 6 92 93 83 25 20 88 75 98 7 81`
	`94 95 81 52 43 19 91 88 0 6 61 19 52 0 69 91 79 87 26 79 26 87 53 38 52 0 6 98 87 58 86`
	`32 79 36 99 25 96 37 97 68 94 84 8 43 78 14 94 83 47 94 90 32 93 44 9 52 1 20 36 46 53`
	`85 53 50 67 21 35 31 40 35`
	
	`Sorting Array using Quick Sort:`
	`0 0 0 1 1 6 6 6 7 8 9 10 14 18 19 19 20 20 21 25 25 26 26 28 31 31 32 32 32 35 35 36 36`
	`36 37 38 40 43 43 44 46 47 49 50 52 52 52 52 53 53 53 54 54 57 57 58 61 61 66 67 68 69`
	`71 75 78 79 79 79 81 81 83 83 84 85 86 87 87 87 88 88 90 91 91 92 93 93 93 94 94 94 94`
	`95 96 96 97 98 98 99 99 99`

Observations	This code sorts the code very fast by using a partition and sorting each sub list. The quick sort has two function, one for sorting and splitting the list, one for doing the other sorting. The partition has three conditions that can be met with the 3rd condition when it's done with its first iteration, then the quick sort function does the rest of the sorting still doing the partition until the list is sorted. This, like the merge sort, has a time complexity of O(n log n) because it uses the same strategy that the merge sort implemented.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Yes, we can sort the left sub list and the right sub list from the partition using other sorting algorithms. This is called a hybrid sorting algorithm and it is simple to code if you know how the partition works and how to apply it on the following sorting algorithms. You would need:

Header File

Partition Function: Use the same algorithm as the partition function used on the quick sort.

Code:

```
5   template <typename T>
6   int partition (T arr[], int low, int high) {
7       int pivot = arr[high];
8       int i = low - 1;
9
10      for (int j = low; j <= high - 1; j++) {
11          if (arr[j] < pivot) {
12              i++;
13              std::swap(arr[i], arr[j]);
14          }
15      }
16      std::swap(arr[i + 1], arr[high]);
17      return i + 1;
18  }
```

Main Sorting Algorithm Function: Use either shell sort or merge sort to sort each sub list. For this example we use shell sort to reduce the amount of code to write.

To output and sort the left side of the list, your end point should be at the pivot (equal to the partition function). For the right side, use pivot + 1 as your starting point.

```
std::cout <<"\nLeft Sub List: " <<std::endl;
for (int x = 1; x < pivot; x++) {
    std::cout <<arr[x] <<" ";
}
```

```
std::cout <<"\nRight Sub List: " <<std::endl;
for (int y = pivot + 1; y < size; y++) {
    std::cout <<arr[y] <<" ";
}
```

For the gap, to the get the size of the left sub list, use pivot – 1. For the right, use size – (pivot + 1)

```
int left = pivot - 1;
for (int gap = left / 2; gap > 0; gap /= 2) {
    for (int i = 0; i < pivot; i++) {
        int leftTemp = arr[i];
        int a = i;
        while (a >= gap && arr[a - gap] > leftTemp) {
            arr[a] = arr[a - gap];
            a -=  gap;
        }
        arr[a] = leftTemp;
    }
}
```

```
int right = size - (pivot + 1);
for (int gap = right / 2; gap > 0; gap /= 2) {
    for (int j = pivot + 1; j < size; j++) {
        int rightTemp = arr[j];
        int b = j;
        while (b >= gap && arr[b - gap] > rightTemp) {
            arr[b] = arr[b - gap];
            b -=  gap;
        }
        arr[b] = rightTemp;
    }
}
```

Code:

```
20  template <typename T>
21  void partitionMergeSort (T arr[], int low, int high, int size) {
22      int pivot = partition(arr, low, high);
23
24      std::cout <<"\nLeft Sub List: " <<std::endl;
25      for (int x = 1; x < pivot; x++) {
26          std::cout <<arr[x] <<" ";
27      }
28      int left = pivot - 1;
29      for (int gap = left / 2; gap > 0; gap /= 2) {
30          for (int i = 0; i < pivot; i++) {
31              int leftTemp = arr[i];
32              int a = i;
33              while (a >= gap && arr[a - gap] > leftTemp) {
34                  arr[a] = arr[a - gap];
35                  a -=  gap;
36              }
37              arr[a] = leftTemp;
38          }
39      }
40      std::cout <<std::endl;
41
42      std::cout <<"\nRight Sub List: " <<std::endl;
43      for (int y = pivot + 1; y < size; y++) {
44          std::cout <<arr[y] <<" ";
45      }
46      std::cout <<std::endl;
47
48      std::cout <<"\nSorted List:" <<std::endl;
49
50      int right = size - (pivot + 1);
51      for (int gap = right / 2; gap > 0; gap /= 2) {
52          for (int j = pivot + 1; j < size; j++) {
53              int rightTemp = arr[j];
54              int b = j;
55              while (b >= gap && arr[b - gap] > rightTemp) {
56                  arr[b] = arr[b - gap];
57                  b -=  gap;
58              }
59              arr[b] = rightTemp;
60          }
61      }
62  }
63  #endif
```

Main Program (using a fixed size of 50 instead of 100):

```
1   #include <iostream>
2   #include <cstdlib>
3   #include <time.h>
4   #include "problem1.h"
5
6   const int max_size = 50;
7   void displayAll (int arr[]);
8
9   int main() {
10      int nums[max_size];
11      srand(time(0));
12      for (int i = 0; i < max_size; i++) {
13          nums[i] = rand() % 50;
14      }
15      std::cout <<"50 randomly generated numbers: " <<std::endl;
16      displayAll(nums);
17      partitionMergeSort(nums, 0, max_size - 1, max_size);
18      displayAll(nums);
19
20  }
21
22  void displayAll (int arr[]) {
23      for (int i = 0; i < max_size; i++) {
24          std::cout <<arr[i] <<" ";
25      }
26      std::cout <<std::endl;
27  }
```

Output:

```
50 randomly generated numbers:
37 46 21 38 20 9 38 1 43 3 35 17 33 12 40 37 5 43 4 46 18 23 4 17 43 10 34 6 20 43 22 1 16 20 34 17 3 26 1 23 47 3 2 29 41 26 19 41 8 12

Left Sub List:
1 3 5 4 4 10 6 1 3 1 3 2 8

Right Sub List:
37 38 43 20 46 18 23 37 17 43 38 34 46 20 43 22 43 16 20 34 17 21 26 35 23 47 17 33 29 41 26 19 41 12 40

Sorted List:
1 1 1 2 3 3 3 4 4 5 6 8 9 10 12 12 16 17 17 17 18 19 20 20 20 21 22 23 23 26 26 29 33 34 34 35 37 37 38 38 40 41 41 43 43 43 43 46 46 47
```

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

The sorting algorithms with the fastest performance are both merge sort and quicksort, because by computing by its time complexity, we get an answer of O(58.60) for both merge sort and quicksort, compared to O(225) for shell sort. The reason that these sorting algorithms have a time complexity of O(n log n) is that log n comes from dividing the list into 2 sub lists, and n comes from the number of times you have to iterate through the list; multiplying n and log n gives us O(n log n).

Conclusion:

The sorting algorithms In conclusion, three more types of sorting algorithms are faster than the aforementioned three sorting algorithms: shell sort, merge sort, and quick sort. Shell sort relies on the gap to sort the list in order, but can be unstable due to the gap. Merge sort uses the divide-and-conquer strategy to sort the list by splitting it into 2 and then into even smaller lists, while quick sort uses a pivot, usually at the end, to compare each if the value is higher than, less than, or equal to the pivot and separate the list into two sub lists after the first iteration.

The procedure showed us how each of the algorithms works using a visualization. It showed how the shell sort uses the gap, merge to split the list into 2, and the quick sort to know where the pivot goes. It didn't show us how to code, so we had to use other websites to give us the code or give some ideas on how to use the code, because some of the sorting algorithms use 2 functions to do the whole sorting.

In the supplementary activity, problem 1 made us use the partition to separate the lists into left and right sublists, then use other sorting algorithms, either merge sort or quicksort, to sort each list. In the code, the shell sort sorting is separate since they both have different endpoints; not all of them end at the max size of the list. In problem 2, we have to determine which sorting algorithm is the simplest using its time complexity and explain why merge sort and quicksort have a time complexity of O(n log n), which is mainly due to the divide-and-conquer strategy.

Overall, my activity was fairly easy, considering that there was only 1 activity to demonstrate through code and the other problem can be solved using diagrams and time complexity. Although I would like to do more implementation of hybrid sorting (combining two sorting algorithms) into a program, since it is more difficult than just doing a normal sorting system, where you need to find a way to combine them.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly and ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: **Grammar Correction and Explanation of the Activity**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Using Grammarly to fix any errors and grammar mistakes that occurred in my supplementary activity and conclusion**
Using ChatGPT to explain how does the activity work, while still doing all the work by myself

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.